CareerNet — Sprint 2 AI Usage Log

Project: CareerNet — Job Search and Application Tracking Platform
Course: SOEN 341 — Software Process
Sprint: Sprint 2 (October 8–26, 2026)
Task Scope: Dany's Frontend Tasks (DA-1 through DA-5)
AI Tool: ChatGPT
Implementation: Dany

AI Usage Entry 1 — DA-1A: Shared API Helper

Date: 16/10/2026
Task: DA-1 — Shared API Helper

Purpose of AI Use

Used ChatGPT for step-by-step guidance to implement a reusable API helper for frontend and backend communication.

Prompt / Request

Asked ChatGPT to explain and guide the implementation of Dany's DA-1 shared API helper, including branch creation, API configuration, token authorization, error handling, and testing.

AI Response

ChatGPT provided TypeScript code for an apiFetch() function and explained how to:

- Configure the backend base URL.
- Include Bearer authentication tokens.
- Handle JSON requests and file uploads.
- Process API errors.
- Handle empty responses and file downloads.
- Document the API configuration in the README.

How AI Was Used

Used the suggested implementation to create src/api/client.ts.

Updated README.md with the required API configuration.

Followed the suggested Git workflow to organize, commit, and submit the feature for review.

Validation

- Successfully ran npm run build.
- Checked Git branch synchronization.
- Reviewed staged changes.
- Committed and pushed the implementation.
- Created Pull Request #96.

Reflection

AI assistance helped me understand how to centralize API requests in one reusable function.

I learned how the frontend sends authentication tokens, handles JSON requests, and supports file uploads.

The helper will be tested with real backend endpoints during backend integration.

AI Usage Entry 2 — DA-1B: Job Listing Page

Date: 16/10/2026

Task: DA-1 — Job Listing Page with Mock Data

Purpose of AI Use

Used ChatGPT to help develop the Job Listing Page using React, TypeScript, mock data, and pagination.

Prompt / Request

Asked ChatGPT to guide me step by step through creating the job types, fictional job data, mock API functions, JobCard component, Jobs Page, routes, and navigation.

AI Response

ChatGPT provided code and implementation guidance for:

- src/types/job.ts
- src/mocks/jobs.ts
- src/api/jobs.ts
- src/components/JobCard.tsx
- src/pages/JobsPage.tsx

- Changes to src/App.tsx
- Changes to src/components/Navbar.tsx

The suggested implementation included 15 fictional jobs, reusable job cards, pagination, loading states, error handling, and protected navigation.

How AI Was Used

Implemented the suggested files in VS Code.

Created the mock job dataset using the categories and locations from Kaila's existing sample-job script.

Connected the page to the mock API functions.

Implemented pagination and integrated the page with CareerNet's existing routing and navigation.

Validation

- Successfully ran npm run build.
- Tested the page using mock login.
- Verified that pagination displayed 10 jobs on the first page and 5 on the second.
- Tested Previous and Next functionality.
- Verified the displayed job information.
- Confirmed protected route behavior.
- Reviewed the staged changes before committing.
- Committed and pushed the implementation.

Reflection

AI assistance helped me understand how to separate frontend components, TypeScript interfaces, mock data, and API functions.

I learned how pagination works and how to implement reusable job cards.

Keeping the mock API separate from the interface will make it easier to connect the page to the real backend during DA-5.

AI Usage Entry 3 — DA-2: Search and Filter Controls

Date: 17/10/2026
AI Tool: ChatGPT
Task: DA-2 — Search and Filter Controls

Purpose of AI Use

Used ChatGPT for step-by-step guidance to implement job searching and filtering in the CareerNet Job Listing Page.

Prompt / Request

Asked ChatGPT to continue Dany's DA-2 frontend task, including branch creation, filter constants, keyword search, location and category filtering, updates to the Jobs Page, testing, and Git workflow.

AI Response

ChatGPT provided implementation guidance and TypeScript/React code to:

- Create src/constants/jobFilters.ts.
- Implement case-insensitive keyword matching.
- Add location and category filtering to the mock Jobs API.
- Apply multiple filters simultaneously.
- Add Search and Clear Filters controls.
- Reset pagination when applying filters.
- Display a friendly message when no jobs match.
- Test all search and filtering behaviours.

How AI Was Used

Implemented the suggested code in VS Code and connected the new search controls to the existing mock Jobs API.

Used the existing job categories and locations to ensure consistency with the backend data.

Adjusted the Clear Filters button's text and background colours to improve readability.

Followed Git workflow guidance to commit, push, and create a Pull Request.

Validation

- Successfully ran npm run build.
- Completed 11 manual tests successfully.

- Verified keyword, location, category, and combined filtering.
- Tested case-insensitive searches and no-results behaviour.
- Confirmed pagination reset and Clear Filters functionality.
- Reviewed Git changes before committing.
- Created Pull Request #98.

Reflection

AI assistance helped me understand how to manage search form state separately from applied filters.

I learned how to implement case-insensitive searching, combine multiple filtering conditions, and apply pagination after filtering.

Manual testing helped confirm that the functionality worked correctly, while improving the Clear Filters button taught me to consider readability and usability alongside functionality.

The filtering logic currently uses mock data and will be connected to the real backend during DA-5.

AI Usage Entry 4 — DA-3: Job Detail Page

Date: 17/10/2026
AI Tool: ChatGPT
Task: DA-3 — Job Detail Page

Purpose of AI Use

Used ChatGPT for step-by-step guidance in developing the Job Detail Page using React, TypeScript, React Router, and mock data.

Prompt / Request

Asked ChatGPT to continue DA-3, including creating the feature branch, implementing the Job Detail Page, reading the job ID from the URL, displaying job information, handling missing and closed jobs, and performing manual testing.

AI Response

ChatGPT provided code and guidance for creating JobDetailPage.tsx and adding the protected /jobs/:id route.

It explained how to use React Router's useParams(), load job details through the existing mock API, display loading and error states, and prevent users from applying to closed jobs.

It also recommended leaving an integration point for the KR-3 Apply Dialog until the relevant feature branches are combined.

How AI Was Used

- Implemented the suggested Job Detail Page in VS Code.
- Connected the page to the existing getJob() function.
- Added dynamic job routing.
- Implemented open, closed, invalid, and missing job handling.
- Added Back to Jobs navigation.
- Followed the suggested manual testing and Git workflow.

Validation

- Successfully ran npm run build.
- Completed seven manual tests.
- Verified different job details and routing.
- Tested closed job handling and missing job messages.
- Confirmed protected route behaviour.
- Checked Git changes before committing.
- Committed, pushed, and created Pull Request #99.

Reflection

AI assistance helped me understand how React Router handles dynamic URLs and how a component can load different information based on a route parameter.

I learned how to manage loading, error, and missing-data states for individual job postings.

I also understood why existing components should be reused rather than duplicated and how to prepare an integration point while dependent feature branches are still awaiting review.

The page currently uses mock data. Real backend integration and the Apply Dialog connection remain pending.

AI Usage Entry 5 — DA-4: My Applications Page

Date: 17/10/2026
AI Tool: ChatGPT
Task: DA-4 — My Applications Page

Purpose of AI Use

Used ChatGPT for step-by-step guidance to build a job application tracking page using React, TypeScript, and mock data.

Prompt / Request

Asked ChatGPT to continue the DA-4 task, including creating the feature branch, defining application data types, generating mock application records, creating status badges, implementing a mock API, building the page, adding navigation, testing, and committing the work.

AI Response

ChatGPT provided implementation guidance and code for:

- The MyApplication interface and five supported application statuses.
- Eight fictional job application records.
- A reusable StatusBadge component.
- The getMyApplications() mock API function.
- The My Applications Page with a table, sorting, loading state, error handling, and empty state.
- The protected /applications route and Navbar link.
- Manual testing and Git workflow.

How AI Was Used

- Followed implementation instructions in VS Code.
- Created and connected the application types, mock data, API function, and React components.
- Implemented the application's status display and table layout.
- Added navigation and protected routing.
- Ran the production build and performed manual browser testing.
- Followed Git instructions to commit, push, and create Pull Request #100.

Validation

- Successfully ran npm run build.
- Completed all seven manual tests.

- Confirmed that eight applications displayed correctly.
- Verified all five application statuses.
- Checked sorting, date information, horizontal scrolling, navigation, and protected access.
- Verified the working tree was clean after committing.
- Submitted Pull Request #100 for review.

Reflection

AI assistance helped me understand how to organize a React feature across types, mock data, API functions, reusable components, and pages.

I learned how to use TypeScript union types for application statuses, implement reusable status badges, and sort data by submission date.

I also learned to keep mock API functions separate from the page so they can be replaced with real backend calls later.

The page currently uses fictional application records. Integrating real user-specific applications and status updates remains future work.